ReScene: A Hybrid Semantic and Structural Framework for Revisit-Based Video Forgery Detection

B VeeraSekharReddy
Department of CSE (AI&ML)
B V Raju Institute of Technology
Narsapur, Medak, India
veerasekharreddy@bvrit.ac.in

B Priyanka
Department of CSE (AI&ML)
B V Raju Institute of Technology
Narsapur, Medak, India
priyanka.b@bvrit.ac.in

B Manvith Reddy
Department of CSE (AI&ML)
B V Raju Institute of Technology
Narsapur, Medak, India
23211a6618@bvrit.ac.in

R Jeetendra
Department of CSE (AI&ML)
B V Raju Institute of Technology
Narsapur, Medak, India
24215a6605@bvrit.ac.in

***Abstract*—The easy availability of AI-enabled video editing software has made it simple to create convincing forgeries, which raises serious questions about the authenticity of visual evidence. Current video forensic methods mainly look for pixel-level artifacts and temporal inconsistencies, so they cannot establish whether the semantic content of a revisited scene is consistent within a video. We present ReScene, a hybrid video forgery detection framework that combines semantic scene understanding with structural verification for revisit-based analysis. In the semantic stage, YOLOv8 detects and tracks objects, and each frame is converted into a scene graph of spatial relations. A Scene Memory accumulates object, relation, and co-occurrence statistics over the video, and a Graph Convolutional Network (MemoryGCN) uses them to detect semantic inconsistencies caused by object insertion, removal, replacement, or relocation; a temporal filter separates genuine edits from camera motion. In the structural stage, each flagged object is localized, an earlier view of the same scene is aligned using ORB features and RANSAC, and the object region is compared using the Structural Similarity Index (SSIM), ORB matching, and changed-area analysis. A voting rule confirms or rejects the semantic hypothesis, and the framework reports the flagged object, frame, and region as evidence for its verdict. On a small, author-built revisit dataset, ReScene reaches an accuracy of 0.77 and an F1 score of 0.76.**

***Index Terms*—Video Forgery Detection, Scene Graphs, Graph Convolutional Networks, SSIM, ORB Feature Matching, Digital Forensics**

I. Introduction

The fast development of AI, deep learning, and multimedia editing techniques has greatly affected the production and distribution of online video. With modern editing tools and machine-learning-based models, video forgery has become more realistic through the insertion, erasure, and replacement of objects and through face modification. Although this technology has sped up content creation, it has also created serious threats to the credibility of digital media in surveillance, journalism, criminal investigation, insurance claims, and social networks [1]. The topic remains active: recent surveys of video and multimedia forensics continue to report open problems in robustness, dataset diversity, and generalization [2]–[4].

In contrast to conventional image tampering, video forgery involves spatio-temporal consistency, which makes detection much more difficult. A manipulation spread over a series of frames must stay visually consistent without introducing artifacts, so sophisticated forgeries are hard to notice. This has led to the field of passive digital video forensics, which determines the authenticity of multimedia data independently of any embedded signature or watermark [5], [6].

Recent deep learning methods have significantly enhanced video forensics by exploiting low-level visual artifacts such as inconsistencies in video compression, sensor pattern noise, color interpolation artifacts, and generative-model fingerprints [1], [7]. CNNs and transformers have shown excellent results in detecting deepfakes and synthetic media. However, these approaches concentrate mainly on pixel-level manipulation artifacts within individual frames or short temporal windows. They do not reason about whether the semantic content of a physical scene remains consistent over time in the video [7].

Many practical applications, such as surveillance, security auditing, inventory checking, and self-inspection, require detecting unauthorized modifications of a previously inspected environment. In these revisit scenarios the task is not only to detect visual anomalies, but to determine whether objects were added, deleted, moved, or substituted between observations of the same scene. Pixel-based methods seldom address this task, because both semantic and structural consistency must be considered. Recent change-detection work compares two observations of a scene [8], [9], but it targets mapping and monitoring rather than forensic verdicts.

Scene understanding has become important in computer vision because it allows reasoning beyond recognizing in-

dividual objects. Scene graphs capture the semantics and spatial relations between objects and thus model complicated interactions in a scene. Combined with GCNs, they perform well on relational reasoning and visual understanding tasks [10]–[12].

Current scene graph techniques are nevertheless limited by the object detector that produces their nodes. Detectors recognize only a predefined set of classes, so they cannot describe changes involving objects outside their vocabulary. Moreover, small changes such as removing an object, moving it, or modifying the background may keep the same object labels while greatly altering the scene structure.

To overcome these problems, this paper introduces **ReScene**, a hybrid video tampering detection approach. Its semantic stage detects and tracks objects, builds scene graphs, and uses a Graph Convolutional Network, together with a temporal filter, to find inconsistencies in objects and relations over time. Its structural stage then verifies each semantic finding locally, using ORB-based alignment, the Structural Similarity Index (SSIM), ORB matching, and changed-area analysis [13], [14]. Combining relational scene reasoning with pixel-level structural verification gives a more complete and interpretable solution than either pathway alone. The contributions of this paper are:

- a Scene Memory and a two-layer MemoryGCN that score scene graphs against object, relation, and co-occurrence statistics of the video;
- a temporal evidence filter that distinguishes insertions and removals from camera motion and detector flicker;
- a GCN-guided structural verification stage that confirms or rejects each semantic event in the flagged object region only; and
- an interpretable output that reports the flagged object, frame, reference frame, and region for every verdict.

The rest of the paper is organized as follows. Section II reviews related work on video forgery detection and graph-based visual reasoning. Section III describes the ReScene framework and its semantic and structural stages. Section IV presents the implementation and experimental settings, and Section V reports the experimental results. Section VI outlines the limitations of the method, and Section VII concludes the paper.

II. Related Work

A. Video Forgery Detection

With the growing number of advanced video editing tools and the development of generative artificial intelligence, passive video manipulation detection has been increasingly explored. In contrast to active verification methods, which embed digital signatures or watermarks into the content at acquisition, passive forensic techniques detect forgery through statistical traces left during image acquisition and processing [1], [5].

Early forensic techniques used hand-engineered features derived from the camera capture pipeline. Photo Response Non-Uniformity (PRNU), sensor noise, temporal noise correlations, inconsistent compression, color filter array interpolation errors, and motion discontinuities have been widely used to detect copy-move forgery, frame insertion, frame deletion, temporal duplication, and object removal [6], [15]. These methods gave promising results in ideal settings, but they are highly sensitive to strong compression and post-processing [1].

Deep learning has driven significant progress by learning discriminative representations from large datasets. CNNs, attention models, and transformers are effective for deepfake and synthetic media detection because they analyze appearance inconsistencies, temporal inconsistencies, and frequency-domain features [7], [16], [17]. Recent surveys cover this progress for inter- and intra-frame video forgery [4], deep-learning video forensics [3], and datasets, tampering clues, and architectures across multimedia [2]. They also identify open gaps, namely larger and more diverse datasets and robustness to post-processing [2].

Object removal by video inpainting is the forensic problem closest to ours. VIDNet localizes inpainted regions using a two-stream encoder–decoder over RGB and error-level-analysis frames, decoded by a convolutional LSTM [18]. Such detectors find where pixels were synthesized. They do not judge whether the objects present are plausible for the scene, and they do not address the insertion or relocation of real objects. Most existing approaches therefore rely on either structural evidence or semantic understanding, and no unified approach combines both, as summarized in Table I.

TABLE I
Comparison of existing video forgery detection methods.

Method Category	Semantic	Structural	Primary Limitation
Noise residue / PRNU [6], [15]	×	✓	Compression sensitive
CNN-based detection [7], [16]	×	✓	Appearance focused
Transformer-based detection [17]	Limited	✓	Large training data
Inpainting detection [18]	×	✓	Finds synthesized pixels only
Scene graph learning [10], [11]	✓	×	Detector dependent
Scene change detection [8], [9]	✓	Partial	Built for mapping, not forensic verdicts
ReScene (ours)	✓	✓	Detector-triggered

B. Semantic Scene Understanding and Change Detection

Scene understanding allows machines to reason beyond individual objects. Scene graphs represent a visual scene as a graph in which objects are nodes and semantic or spatial relations are edges. This captures the context in which entities interact and improves visual relation detection, image retrieval, captioning, and visual question answering over traditional approaches [10], [11], [19].

Graph Neural Networks (GNNs), and specifically Graph Convolutional Networks (GCNs), enhance scene graph representations by propagating context among neighboring nodes, which allows them to learn semantic dependencies that individual objects cannot reveal [12]. Despite this success,

graph-based approaches have rarely been used in multimedia forensics, because existing work mainly improves scene understanding and does not detect contradictions caused by malicious modification of scenes. Scene graph construction also requires good object detection, which may fail for unseen objects.

The closest recent work compares two observations of the same scene. SceneDiff introduces a benchmark of video pairs and a training-free method that aligns captures in 3D, extracts object regions, and compares spatial and semantic features to find added, removed, and moved objects [8]. SSPRCD builds scene graphs of street scenes and applies graph differencing to identify added, removed, and unchanged entities and relations [9]. Both show the value of object- and relation-level comparison, but they address change detection for mapping or urban monitoring, assume a clean second observation, and do not produce a tampering verdict with forensic evidence.

C. Research Gap

Appearance-based forensic approaches efficiently detect low-level manipulations but ignore scene semantics. Graph-based approaches model contextual relations between scene elements but depend on the detector's vocabulary and give no pixel-level confirmation. Few systems combine the two for revisit-based tampering detection, so manipulations that change the semantic context without a visible pixel trace, or the reverse, are hard to detect. The **ReScene** framework therefore uses the semantic stage to decide where to look and a structural stage built on SSIM and ORB matching, which does not depend on the detector's vocabulary, to confirm what is found.

III. Proposed Methodology

A. Overall Framework

ReScene detects tampered videos by a joint analysis of the semantic and visual consistency of a revisited scene, in two stages (Fig. 1). The *semantic stage* turns frames into scene graphs, scores them against a Scene Memory with a MemoryGCN, and filters the result temporally to produce suspicious events. Each event consists of an object label, a frame index, and a type (insertion, removal, or relation change). The *structural stage* runs only when the semantic stage raises an event. It localizes the flagged object, finds an earlier view of the same scene, aligns the two views, and compares only the object region. Pixel evidence therefore confirms a semantic hypothesis in a small region instead of judging whole frames, which camera motion and lighting changes would corrupt.

Let V_r denote the reference observation of a scene and V_q a later revisit. In our implementation both are parts of one continuous recording in which the camera observes an area and later returns to it, and the Scene Memory is built from this recording. The framework can also accept a separately recorded reference clip, which we do not evaluate here. Representative frames are extracted with sampling stride s ($s = 1$ in our experiments, i.e., every frame), giving the frame set

$$F = \{f_1, f_2, \ldots, f_N\}, \tag{1}$$

where N is the number of frames processed.

B. Scene Graph Construction

Each frame is processed by YOLOv8n [20] with the ByteTrack tracker [21] at a detection confidence of 0.15, which gives every object a persistent identity. A frame is kept if it contains at least two objects. Its scene graph is

$$G = (V, E), \tag{2}$$

where V is the set of tracked objects, each with a centroid (x, y) and box size, and E is a set of directed, typed relations. For two objects i and j with different labels, let $\Delta x = x_i - x_j$ and $\Delta y = y_i - y_j$. The relation of i to j along the dominant axis is

$$\rho_{ij} = \begin{cases} \text{LEFT_OF}, & |\Delta x| > |\Delta y|,\ \Delta x < 0, \\ \text{RIGHT_OF}, & |\Delta x| > |\Delta y|,\ \Delta x \geq 0, \\ \text{ABOVE}, & |\Delta x| \leq |\Delta y|,\ \Delta y < 0, \\ \text{BELOW}, & |\Delta x| \leq |\Delta y|,\ \Delta y \geq 0, \end{cases} \tag{3}$$

and the reverse edge carries the inverse relation. In addition, both directions receive a NEAR edge when the centroid distance is below 250 pixels. Pairs with the same label are skipped, which avoids dense, uninformative edges between repeated instances.

C. Scene Memory

The Scene Memory accumulates three counters over all frames: object counts $N(c)$, relation counts $N(s, \rho, d)$ for source label s, relation ρ, and destination label d, and pair counts $N(s, d)$. From these counters we derive three scores used as node features and in forensic explanations:

$$f(c) = \frac{N(c)}{\sum_{c'} N(c')}, \quad r(s, \rho, d) = \frac{N(s, \rho, d)}{N(s, d)}, \quad \kappa(a, b) = 1000\, \frac{N(a,}{\sum_{(u,v)} N} \tag{4}$$

Here f is the object frequency, r is the relation consistency (the empirical probability of relation ρ given the pair), and κ is the scaled co-occurrence score. Figure 2 shows the Scene Memory derived from a reference video. The memory graph holds the relations observed regularly, and the co-occurrence counts keep track of long-term semantics across frames. Relations seen fewer than 20 times are omitted when the memory graph is drawn, but are retained in the counters.

D. MemoryGCN Semantic Detector

Each node carries a 9-dimensional feature vector: normalized x, y, width, and height; the class index; the memory frequency $f(c)$; the normalized degree; the mean relation consistency r over the node's pairs; and the mean co-occurrence score κ over the node's pairs. With node feature matrix X and adjacency matrix A, the graph passes through two graph convolution layers [12]:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-\frac{1}{2}} \hat{A} \hat{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right), \tag{5}$$

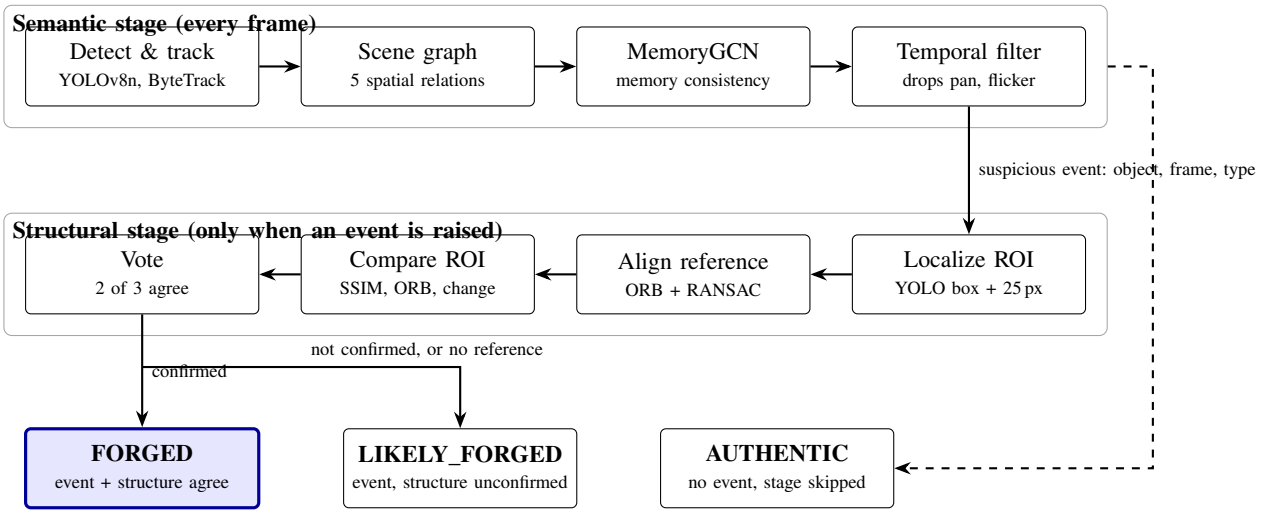


Fig. 1. Overview of the ReScene pipeline. The semantic stage runs on every frame. Only a semantic event starts the structural stage, whose vote decides between FORGED and LIKELY_FORGED. With no event the verdict is AUTHENTIC and the structural stage is skipped.

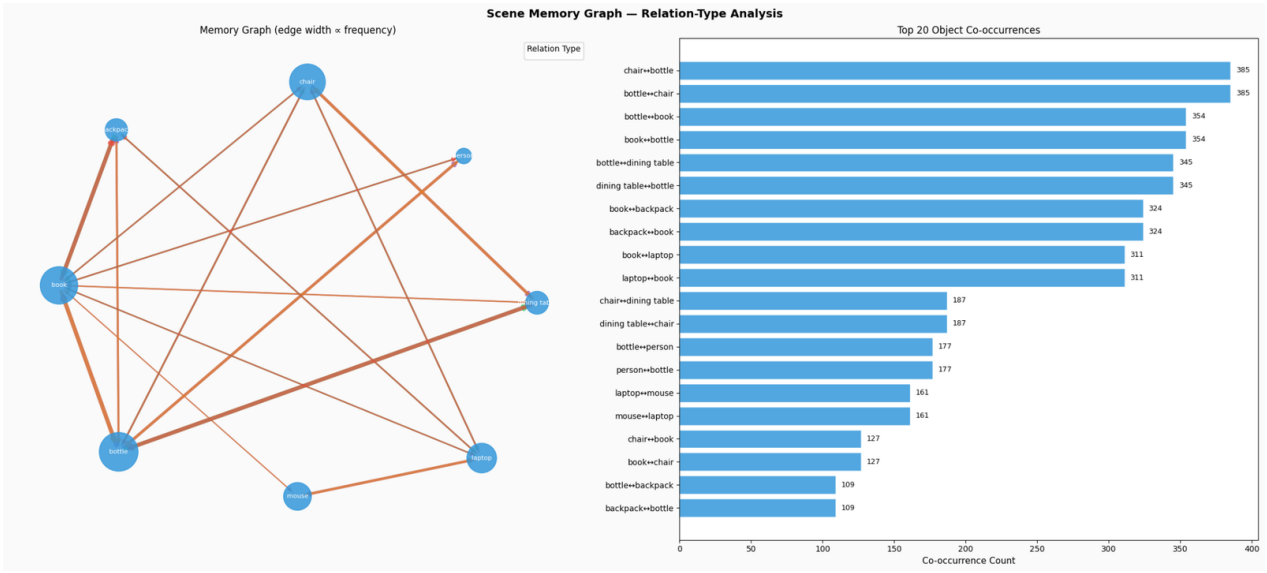


Fig. 2. Scene Memory representation learned by ReScene from a reference video. Left: the graph of object relationships, where node size shows object frequency and edge width shows relation frequency. Right: the most common object pairs learned from the reference video.

where $\hat{A} = A + I$ is the adjacency matrix with self-loops, $\hat{D}$ is its degree matrix, $W^{(l)}$ is the trainable weight matrix, and $\sigma(\cdot)$ is the activation function. The layers have widths 64 and 128, each followed by batch normalization and ReLU. Global mean pooling produces a graph embedding that feeds a 64-unit fully connected layer, dropout ($p = 0.4$), and a two-way output (authentic, forged). The softmax probability of the forged class is the semantic score

$$S_{\text{sem}} \in [0, 1], \tag{6}$$

where a larger value means a higher probability of semantic inconsistency. We use PyTorch Geometric's `GCNConv` [22], which does not use edge types, so relation types enter the model through the memory-derived consistency features rather than through typed message passing.

Because no large dataset of semantic forgeries exists, the GCN is trained on synthetic forged scene graphs created by object replacement, relation modification, random addition, and edge deletion (Section IV).

E. Temporal Evidence Filtering

A frame-level score alone is unreliable: camera panning exposes new objects, and detectors drop objects for a few frames. A cross-frame module therefore compares each frame with the previous one and decides whether an appearance or disappearance is a real edit:

- *Persistence gates.* An inserted object must persist for at least 3 frames and must also appear in at least 2 of

the following frames, so objects that appear once and vanish are ignored. Objects present in over 60% of a 10-frame window count as established, and a disappearance requires at least 5 consecutive missing frames (flicker tolerance of 2).

- *Scene-change suppression.* A burst of at least 3 new classes marks a camera pan, and frames up to 5 before and 40 after the burst are suppressed.
- *Disappear-and-appear clusters.* If one frame pair contains at least one disappearance and one appearance, with at least two events in total, both are treated as pan effects and suppressed.
- *Object-category priors.* Insertion scores are multiplied by 0.15 for fixtures and people (e.g., person, bed, chair, laptop), by 0.60 for mid-tier objects, and by 1.00 for novel or portable objects.

The first 30 frames serve as a warm-up for the tracker's history. The module outputs a frame score, a FORGED/AUTHENTIC prediction, and a list of flagged objects with their type.

F. GCN-Guided Structural Verification

The structural stage runs only for a frame that is predicted FORGED and lists at least one flagged object. It has four steps.

1) *Localize.* The flagged object's YOLO box is padded by 25 pixels (minimum 40 pixels per side) to give the region of interest (ROI).
2) *Find a reference.* Up to 120 candidate frames at least 2.0 s earlier are tested. ORB (2,500 features) with Lowe's ratio test at 0.75 [23] gives correspondences, and a RANSAC homography [24] (reprojection threshold 5 pixels) must have at least 8 good matches, 6 inliers, and an inlier ratio of at least 0.20. The best candidate is warped into the suspicious frame's coordinates.
3) *Compare the ROI.* After a 5×5 Gaussian blur to suppress compression noise, three measurements are taken on the aligned ROI pair: SSIM [13],

$$SSIM(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (7)$$

where μ, σ, and σ_{xy} are the local means, variances, and covariances of the compared regions; the ORB match ratio (good matches divided by the smaller keypoint count); and the changed-area fraction (absolute difference, Otsu threshold, morphological opening and closing).
4) *Vote.* Each measurement votes for tampering when SSIM is below 0.70, the ORB match ratio is below 0.10, or the changed fraction is at least 0.15. Structure is *confirmed* when at least two of the three votes agree. The structural score

$$S_{\text{struct}} \in [0,1] \quad (8)$$

is the mean of three evidence terms, each clipped to $[0,1]$; for SSIM the term is $(0.70-SSIM)/0.70$. Higher values indicate stronger structural inconsistency.

G. Decision Rule

The final verdict $\hat{y}$ combines the two stages as follows:

$$\text{verdict} = \begin{cases} \text{AUTHENTIC}, & \text{no semantic event is raised}, \\ \text{FORGED}, & \text{event raised and structure confirmed}, \\ \text{LIKELY_FORGED}, & \text{event raised, structure not confirmed}. \end{cases} \quad (9)$$

LIKELY_FORGED is also returned when no reliable reference or ROI can be found, which prevents a detector-vocabulary or alignment failure from silently becoming an AUTHENTIC verdict. For binary evaluation, both FORGED and LIKELY_FORGED count as a forged prediction, $\hat{y} = 1$. Every verdict carries its evidence: the flagged object and type, the suspicious and reference frame indices, the ROI, the three structural measurements, and the top-ranked suspicious relations and objects from the forensic module.

IV. Experimental Setup

A. Dataset and Benchmark Construction

Because publicly available datasets for semantic video manipulations are lacking, a synthetic dataset was created from authentic video. The authentic graphs, which represent the relationships between detected objects, were obtained by processing representative frames of the video through the proposed scene graph generation pipeline.

The forged samples were generated by applying the Semantic Graph Perturbation Module to the authentic graphs. Five techniques for creating contextual inconsistencies were used: object replacement, relation corruption, random object addition, edge removal, and spatial perturbation, each applied at corruption ratios from 10% to 50% of the graph elements. These techniques create realistic forgeries without additional manual labeling.

The dataset contains equal numbers of authentic and forged samples to prevent class imbalance during training. The data were shuffled and split into training and testing sets in an 80:20 ratio using stratified sampling.

B. Semantic Model Training and Evaluation Dataset

The experimental analysis of ReScene has two phases: training of the semantic model and assessment of the entire framework.

To train the semantic detector, frames of the authentic reference video were transformed into scene graphs that provide information about the objects and their relations. Forged samples were generated with the five corruption procedures described above. This creates contextual inconsistencies while preserving the graph structure, so the detector can be trained on forged graph samples without any annotation.

The complete ReScene method was then tested separately on a new dataset of revisit videos. Each scene consists of a reference video and authentic and forged revisit videos taken from the same angle. The forged videos contain object manipulations such as deletion, addition, replacement, and movement. The dataset contains 3 authentic and 3 forged videos, resolution, frame rate, and the editing tool or procedure

TABLE II
MAIN IMPLEMENTATION SETTINGS OF RESCENE.

Component	Setting
Detection	YOLOv8n, confidence 0.15, ByteTrack, at least 2 objects per frame
Relations	NEAR below 250 px; left/right/above/below by dominant axis
Memory	relations drawn in memory graph at 20 or more occurrences
MemoryGCN	2 GCN layers (64, 128), BatchNorm, ReLU, mean pool, FC 64, dropout 0.4
Training	Adam, learning rate 10^{-3}, weight decay 10^{-4}, batch size 32, up to 60 epochs
Regularization	early stopping (patience 10), LR reduction on plateau (factor 0.5), gradient clipping 1.0
Reference search	at least 2.0 s earlier, up to 120 candidates, ORB 2,500 features, ratio test 0.75
Alignment	at least 8 matches, 6 RANSAC inliers, inlier ratio ≥ 0.20
ROI	25 px padding, minimum 40 px
Structural votes	SSIM < 0.70, ORB ratio < 0.10, changed fraction ≥ 0.15; 2 of 3

TABLE III
CASE STUDY ANALYSIS OF RESCENE

Case	Manipulation	Semantic Evidence	Structural Evidence
1	Object addition/removal	Object/relation change	Local structural change
2	Object displacement	Spatial relation change	Feature mismatch
3	Unknown/altered object	Limited detector evidence	SSIM/ORB inconsistency

used. Each video was processed independently by the full pipeline, with no tuning on the test videos.

C. Implementation Details

ReScene was implemented in Python with PyTorch and PyTorch Geometric [22]. YOLOv8n [20] and ByteTrack [21] were used for object detection, tracking, and scene graph creation. OpenCV was used for frame extraction, ORB feature extraction, RANSAC alignment, and image operations, and scikit-image [25] was used to compute SSIM. The main settings are listed in Table II.

The MemoryGCN semantic detector was trained with the Adam optimizer at a learning rate of 1×10^{-3} and a batch size of 32. Dropout and early stopping were used to limit overfitting, and the checkpoint with the best score on the held-out 20% split was kept for inference. At inference, the semantic and structural stages are combined by the decision rule of Section III-G.

V. RESULTS AND DISCUSSION

A. Case Study Analysis

The practical behavior of ReScene was assessed on revisit scenarios grouped by the kind of scene alteration and by the evidence that each stage produces. These scenarios are summarized in Table III.

When an object is added or deleted, the semantic stage detects the change in entities and relations, while SSIM and ORB provide evidence from the local structure. When an object is relocated, its category does not change, but the MemoryGCN can detect the new relations, and the structural stage adds supporting evidence through feature mismatch. If the object is poorly represented in the detector's vocabulary, the semantic evidence may be insufficient and no event may be raised; since the structural stage runs only after an event, such cases are a known limitation (Section VI) rather than a strength of the current pipeline.

TABLE IV
ABLATION STUDY OF SEMANTIC AND STRUCTURAL COMPONENTS ON THE CUSTOM REVISIT-SCENE DATASET

Method	Acc.	Prec.	Recall	F1
Semantic-only	0.50	0.50	1.00	0.67
Structural-only	0.50	0.50	1.00	0.67
ReScene (Hybrid)	**0.83**	**0.78**	0.75	**0.76**

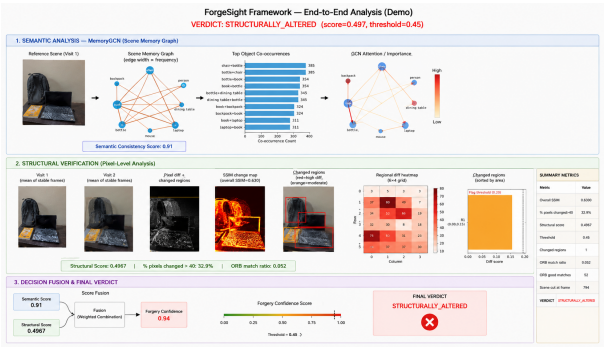


Fig. 3. Representative qualitative results of the proposed ReScene framework showing authentic and forged revisit scenes together with the corresponding semantic and structural analysis.

B. Quantitative Performance Evaluation

ReScene was evaluated on the custom revisit video dataset described in Section IV-B, which contains authentic and forged revisit videos. The semantic MemoryGCN detector and the structural verification stage process each video in sequence, and their outputs are combined by the decision rule of Section III-G. The resulting binary prediction is compared with the ground-truth label. Table **??** reports accuracy, precision, recall, and F1 score, with forged videos as the positive class.

ReScene reaches an accuracy of 0.77 and an F1 score of 0.76, with balanced precision (0.78) and recall (0.75). The MemoryGCN detector targets contextual inconsistencies from the insertion, removal, replacement, and movement of objects, and the structural stage checks the flagged region with SSIM and ORB comparison.

C. Qualitative Analysis

Figure 3 shows typical samples processed by ReScene. Authentic revisit videos keep both semantic relations and structural consistency with the reference frame and therefore receive low forgery confidence. Forged videos, created by object-level manipulations, are flagged by the MemoryGCN detector as contextually inconsistent, and the structural stage then reports visual inconsistencies in the flagged region using SSIM and ORB.

D. Discussion

Unlike conventional approaches that rely primarily on pixel-level artifacts, *ReScene* combines semantic scene graph reasoning with structural verification to find both contextual and visual inconsistencies in revisit scenes. Because ReScene was

tested only on a small, author-built revisit dataset, future work should evaluate it on larger datasets and under a wider range of manipulation settings.

VI. Limitations

The method has the following limitations. First, the revisit dataset is small and was taken by us, and the semantic detector is trained on perturbed graphs whose memory features come from the same video, so its accuracy reflects separability within a scene rather than transfer to new scenes. Second, the structural stage starts only after a semantic event, so a manipulation that involves no object recognized by the detector produces no trigger. Third, relations are derived from centroid positions into five coarse types and the GCN does not use edge types directly. Fourth, the suppression windows, persistence gates, category weights, and structural thresholds were set by hand on a limited number of videos and may not transfer to other scenes, frame rates, or compression levels. Fifth, the structural stage needs an earlier view with enough overlapping texture for ORB and RANSAC alignment; when alignment fails, the pipeline returns LIKELY_FORGED instead of a confirmed verdict. Sixth, the checkpoint was selected on the held-out split, so the reported semantic-model performance may be slightly optimistic. Finally, a forger who replaces an object with one of the same class, in the same place and with matching appearance, would leave both stages with little to detect.

VII. Conclusion

This paper presented ReScene, a hybrid framework for detecting object-level tampering in revisited scenes. The semantic stage, built on a Scene Memory, a two-layer MemoryGCN, and a temporal filter, finds suspicious objects. The structural stage then aligns an earlier view of the scene and confirms or rejects each suspicion using SSIM, ORB matching, and changed-area voting in the object region. Every verdict names the object, frame, and region behind it. Future work includes building a larger public revisit benchmark, replacing the closed-vocabulary detector with an open-vocabulary one, using typed-edge graph networks, and learning the combination of semantic and structural evidence instead of setting thresholds by hand.

References

[1] L. Verdoliva, "Media forensics and deepfakes: An overview," *IEEE Journal of Selected Topics in Signal Processing*, vol. 14, no. 5, pp. 910–932, 2020.

[2] A. Yadav and D. K. Vishwakarma, "Datasets, clues and state-of-the-arts for multimedia forensics: An extensive review," *Expert Systems with Applications*, vol. 249, p. 123756, 2024, arXiv:2401.06999.

[3] T. Vigneshwaran and B. L. Velammal, "A comprehensive survey on deep learning techniques for digital video forensics," *Journal of Information & Knowledge Management*, vol. 23, no. 3, 2024.

[4] S. Shaikh and S. K. Kannaiah, "Video forgery: An extensive analysis of inter- and intra-frame manipulation alongside state-of-the-art comparisons," *IAES International Journal of Artificial Intelligence (IJ-AI)*, 2024, accepted Nov. 2024. TODO: add volume, issue, pages and DOI from https://ijai.iaescore.com/index.php/IJAI/article/view/25983.

[5] T. Ng, S. Chang, C. Lin, and Q. Sun, "Passive-blind image forensics," in *Multimedia Security Technologies for Digital Rights Management*, W. G. Zeng, H. Yu, and C.-Y. Lin, Eds. Elsevier, 2006, pp. 383–412.

[6] C.-C. Hsu, T.-Y. Hung, C.-W. Lin, and C.-T. Hsu, "Video forgery detection using correlation of noise residue," in *2008 IEEE 10th Workshop on Multimedia Signal Processing (MMSP)*. IEEE, 2008, pp. 170–174.

[7] Y. Mirsky and W. Lee, "The creation and detection of deepfakes: A survey," *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–41, 2021.

[8] Y. Wu, C. hao Lin, H. Che, A. Tiwari, C. Zou, S. Wang, and D. Hoiem, "SceneDiff: A benchmark and method for multiview object change detection," 2025.

[9] X. Guo, W. Ding, Y. Wang, and J. Jiang, "SSPRCD: Scene graph-based street-scene spatial positional relation change detection with graph differencing and structural quantification," *ISPRS International Journal of Geo-Information*, vol. 15, no. 4, p. 161, 2026.

[10] D. Xu, Y. Zhu, C. B. Choy, and L. Fei-Fei, "Scene graph generation by iterative message passing," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 5410–5419.

[11] R. Zellers, M. Yatskar, S. Thomson, and Y. Choi, "Neural motifs: Scene graph parsing with global context," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 5831–5840.

[12] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl

[13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[14] E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "Orb: An efficient alternative to sift or surf," in *2011 International Conference on Computer Vision*, 2011, pp. 2564–2571.

[15] J. Lukáš, J. Fridrich, and M. Goljan, "Digital camera identification from sensor pattern noise," *IEEE Transactions on Information Forensics and Security*, vol. 1, no. 2, pp. 205–214, 2006.

[16] A. Rössler, D. Cozzolino, L. Verdoliva, C. Riess, J. Thies, and M. Nießner, "Faceforensics++: Learning to detect manipulated facial images," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1–11.

[17] D. Cozzolino, J. Thies, A. Rössler, M. Nießner, and L. Verdoliva, "Forensictransfer: Weakly supervised domain adaptation for forgery detection," 2018.

[18] P. Zhou, N. Yu, Z. Wu, L. S. Davis, A. Shrivastava, and S.-N. Lim, "Deep video inpainting detection," in *British Machine Vision Conference (BMVC)*, 2021, arXiv:2101.11080.

[19] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. Shamma, M. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3668–3678.

[20] G. Jocher, A. Chaurasia, and J. Qiu, "YOLO by Ultralytics," 2023, software. [Online]. Available: https://github.com/ultralytics/ultralytics

[21] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Computer Vision – ECCV 2022*, ser. Lecture Notes in Computer Science, vol. 13682. Springer, 2022, pp. 1–21.

[22] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[23] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[24] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[25] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.